LAB 4 — VIBECODING

Q*bert Game: AI-Assisted Debugging and Feature Implementation

Student Name: Sateesh E

Course: Software Engineering Lab

Assignment: Lab 4 — VibeCoding

Project: Q*bert Repair Lab

Programming Language: Python

Library: Pygame

Repository: https://github.com/Sateesh5298/PES1UG24AM397_LAB

1. Objective

The objective of this lab was to use an AI coding assistant to identify and fix a deliberate bug in a Python game and implement three additional features. The project uses Pygame to create a Q*bert-style game in which the player moves across a pyramid of cubes.

The assignment involved understanding existing code, writing targeted prompts, reviewing suggested changes, testing the game, recording before-and-after gameplay, and pushing the project to GitHub.

2. Initial Project Understanding

The original project contained a Q*bert-style game with the following components:

- An isometric pyramid made of cubes.
- A player who jumps diagonally between cubes.
- Cube painting and score updates.
- Enemies that chase the player or move down the pyramid.
- Lives, levels, and win/lose conditions.
- One deliberate pyramid projection bug.
- Three functions intended for additional features.

The main objective was to correct the projection bug and improve the game without unnecessarily changing its existing architecture.

3. Task 1 — Fix the Pyramid Projection Bug

Prompt given to the AI assistant

I am working on a Python Pygame Q*bert game. Inspect the `cube_center(row, col)` function and identify the cause of the pyramid projection bug.

The pyramid should appear as a symmetric triangle, but the current coordinate calculation causes the rows to shift incorrectly.

Fix the arithmetic used to calculate the horizontal position. Explain the difference between integer division and true division and why the appropriate calculation fixes the projection.

Make only the changes required for this task. Preserve the existing controls, scoring, and gameplay logic. Explain how to test the corrected pyramid.

AI-assisted analysis

The position of each cube is calculated using its row and column coordinates. Incorrect arithmetic in the horizontal offset can cause rows to be positioned incorrectly, making the pyramid appear skewed.

Integer division (`//`) discards the fractional part of a result, while true division (`/`) preserves fractional values. The correct choice depends on the intended coordinate calculation.

The correction should ensure that each row is centered relative to the row above it and that the pyramid retains its triangular symmetry.

Testing procedure

1. Run `py game.py`.
2. Inspect the pyramid from the first frame.
3. Confirm that the rows are arranged symmetrically.
4. Test the arrow keys to verify that player movement still works.

Commit recorded

```
Fix pyramid projection
```

4. Task 2 — Implement Level-Specific Cube Palettes

Prompt given to the AI assistant

Implement the `cube_palette(level)` function in `game.py`.

The function must return a list containing exactly `TARGET + 1` RGB color tuples, currently three colors, representing the different cube painting stages. It may return `None` to use the default palette.

Use different, visually distinct palettes for different levels. Every RGB channel must be an integer between 0 and 255.

Preserve the existing painting logic, score calculation, and gameplay. Do not implement the other two empty functions in this task. Explain the implementation and how to test it.

AI-assisted analysis

The game uses a palette to determine the colors displayed for different cube stages. A level-dependent palette can make progression visually clearer and distinguish one level from another.

The implementation should return valid RGB tuples and provide exactly three colors for the current configuration.

Testing procedure

1. Run `py game.py`.
2. Observe the cube colors as the player jumps onto cubes.
3. Verify that the three painting stages use the intended colors.
4. Verify that different levels produce different palettes.

Commit required

```
Add level-specific cube palettes
```

5. Task 3 — Implement the Cube-Completion Feature

Prompt given to the AI assistant

Implement the `on_cube_completed(cell)` function in `game.py`.

The function is called when a particular cube first reaches its target painting stage. The function receives the completed cell as a `(row, col)` tuple.

Implement a brief visual flash on the completed cube or an additional score bonus. Use the existing game architecture and avoid triggering the effect on every subsequent hop onto an already completed cube.

Do not change the existing 25-point painting score unless required for the chosen bonus. Explain the changes and provide a testing procedure.

AI-assisted analysis

The cube-completion callback provides an opportunity to make successful gameplay more noticeable. A completion effect should occur when the cube reaches its target stage, not every time the player lands on it afterward.

The implementation must respect the existing painting logic and avoid repeated completion bonuses for the same completion event.

Testing procedure

1. Run the game.
2. Jump onto a cube until it reaches its final painting stage.
3. Observe the completion effect or bonus.
4. Check that the effect does not repeatedly trigger when revisiting the completed cube.

Commit required

```
Add cube completion effect
```

6. Task 4 — Implement the Bonus-Life Threshold

Prompt given to the AI assistant

Implement `bonus_life_threshold()` in `game.py`.

The function takes no arguments and returns an integer score threshold or `None` to disable bonus lives.

Use a threshold of 1000 points. The existing game logic maintains the `self.bonus_awarded` bookkeeping, so use the existing mechanism rather than implementing a separate tracking system.

Preserve the current score and life systems. Explain the implementation and how to test the bonus-life behavior.

AI-assisted analysis

A bonus-life threshold rewards players for achieving higher scores. With a threshold of 1000, the game should award an additional life when the score crosses each new threshold, according to the existing bookkeeping logic.

The feature should avoid repeatedly awarding a life for the same threshold.

Testing procedure

1. Run the game.
2. Accumulate points through cube painting.
3. Verify the bonus-life behavior at the threshold.
4. Confirm that crossing a new threshold awards a life only once.
5. Check that ordinary scoring and lives remain functional.

Commit required

```
Implement bonus life threshold
```

7. Final Game Verification

The following checks should be completed before submission.

Test	Expected result
Game startup	Game window opens without errors
Pyramid projection	Pyramid is symmetric and fully visible
Player movement	Arrow keys move the player
Cube painting	Valid landings change cube stages
Scoring	Painting awards the expected points
Lives	Player starts with lives remaining
Enemy behavior	Enemies behave as intended
Level palettes	Different levels use different palettes
Completion effect	Effect triggers on cube completion
Bonus lives	New score thresholds award additional lives
Reset	R resets the game
Win/lose conditions	Game handles level completion and lost lives

Evidence note: These are the required verification checks. Mark each test as passed only after observing the result in the actual game.

8. Before-and-After Gameplay Recordings

Before video

The original gameplay was recorded before modifying the code.

Filename: `videos/before.mp4`

Purpose:

- Show the original game.
- Capture the original projection problem.
- Establish a baseline for comparison.

After video

The final gameplay was recorded after implementing and testing the fixes and features.

Filename: `videos/after.mp4`

Purpose:

- Show the corrected pyramid.
- Demonstrate gameplay and cube painting.
- Show an updated feature when possible.

Both videos should be at least 10 seconds long and should show actual gameplay.

9. GitHub Submission

The project was committed and pushed to the personal lab repository.

Repository URL:

https://github.com/Sateesh5298/PES1UG24AM397_LAB

The Git workflow used the following commands:

```
git status
git add -A
git commit -m "Complete Lab 4 Qbert deliverables"
git log --oneline
git push origin main
```

The successful push was verified when Git reported that the local branch was up to date with `origin/main` and the working tree was clean.

The recorded commit history includes:

- `Add Lab 4 Qbert starter project`
- `Fix pyramid projection`
- `Complete Lab 4 Qbert deliverables`

The handout also requires separate commits for each coding task. Confirm the task-specific commit history before final submission.

10. Problems Encountered and Resolution

Problem 1: Running the game

The game was launched from Windows Command Prompt using:

```
py game.py
```

Pygame initialized and opened the game window.

Problem 2: Git staging and committing

The project contained staged changes, unstaged modifications, and an untracked after-video. These were reviewed and staged before committing.

Problem 3: Publishing to GitHub

The project was pushed using:

```
git push origin main
```

Git reported `Everything up-to-date`, and the working tree was subsequently reported clean.

Problem 4: Final gameplay verification

The game window opened, but the displayed pyramid appeared clipped at the right edge and the HUD showed zero lives. These observations require further investigation before claiming that the game passes all final tests.

11. Conclusion

This lab demonstrates the use of AI-assisted coding to understand an existing program, investigate a graphical projection bug, plan feature implementations, and organize code changes with Git.

The key lesson is that AI-generated code must be reviewed and tested rather than accepted without verification. A successful program launch and GitHub push do not, by themselves, prove that all gameplay requirements have been satisfied.

Before submission, the final code, gameplay recordings, exported conversation, and task-specific commit history must all be checked against the lab handout.